Understanding normalizing flows from a mathematical view

Pipi Hu

Beijing Institute of Mathematical Sciences and Applications

November 8, 2025

Feynman's words

What I cannot create, I do not understand.

Outline

Normalizing: Change of Variables Theorem

Given a bijective transformation $T : \mathbb{R}^d \to \mathbb{R}^d$ with inverse T^{-1}, and probability densities $p_X(x)$ and $p_Y(y)$, we have:

$$p_Y(y) = p_X(T^{-1}(y)) \left| \det \frac{\partial T^{-1}(y)}{\partial y} \right| \tag{1}$$

where $\frac{\partial T^{-1}(y)}{\partial y}$ is the Jacobian matrix of the inverse transformation.

- Key insight: We can transform simple distributions into complex ones
- Challenge: Computing determinants is computationally expensive
- Solution: Design transformations with tractable Jacobians

Proof of the Change of Variables Formula (Part 1)

Theorem: For bijective $T : \mathbb{R}^d \to \mathbb{R}^d$ with inverse T^{-1},

$$p_Y(y) = p_X(T^{-1}(y)) \left| \det \frac{\partial T^{-1}(y)}{\partial y} \right| \tag{2}$$

Proof

Let $A \subset \mathbb{R}^d$ be any measurable set. Then:

$$\int_A p_Y(y)dy = \mathbb{P}(Y \in A) = \mathbb{P}(X \in T^{-1}(A)) \tag{3}$$

The key insight is that we can relate the probability measures through the transformation T.

Proof of the Change of Variables Formula (Part 2)

Proof (Cont)

Using the change of variables formula for integrals:

$$\mathbb{P}(X \in T^{-1}(A)) = \int_{T^{-1}(A)} p_X(x)dx = \int_A p_X(T^{-1}(y)) \left|\det \frac{\partial T^{-1}(y)}{\partial y}\right| dy \tag{4}$$

Since this holds for any measurable set A, we have:

$$p_Y(y) = p_X(T^{-1}(y)) \left|\det \frac{\partial T^{-1}(y)}{\partial y}\right| \tag{5}$$

Key insight: The Jacobian determinant accounts for how the transformation changes volumes in the space.

Remark

This change of variables formula is fundamental to all normalizing flows and ensures that probability mass is conserved during transformations.

Flows: Neural blocks for invertible transformations

A normalizing flow is a sequence of invertible transformations:

$$\mathbf{x}_0 \xrightarrow{T_1} \mathbf{x}_1 \xrightarrow{T_2} \mathbf{x}_2 \xrightarrow{T_3} \cdots \xrightarrow{T_K} \mathbf{x}_K \tag{6}$$

The probability density transforms as:

$$\log p(\mathbf{x}_K) = \log p(\mathbf{x}_0) + \sum_{k=1}^{K} \log \left| \det \frac{\partial T_k}{\partial \mathbf{x}_{k-1}} \right| \tag{7}$$

Goal: Learn $T_1, T_2, \ldots, T_K$ such that $p(\mathbf{x}_K)$ matches the data distribution.

Outline

Autoregressive Transformations

An autoregressive transformation processes variables in order:

$$y_i = \tau(x_i; \mathbf{h}_i) \tag{8}$$

where $\mathbf{h}_i = h_i(x_1, x_2, \ldots, x_{i-1})$ is a function of previous variables.
Key properties:

- Triangular Jacobian: $\frac{\partial y_i}{\partial x_j} = 0$ for $j > i$
- Efficient determinant: $\det J = \prod_{i=1}^{d} \frac{\partial y_i}{\partial x_i}$
- Easy inverse: Can be computed sequentially

Masked Autoregressive Flow (MAF)

MAF uses neural networks with masking to ensure autoregressive structure:

$$y_i = x_i \odot \exp(\alpha_i) + \beta_i \tag{9}$$

where α_i, β_i are outputs of a masked neural network.
Masking strategy:

- Input mask: Only allow connections from $x_1, \ldots, x_{i-1}$ to α_i, β_i
- Ensures $\frac{\partial y_i}{\partial x_j} = 0$ for $j \geq i$
- Jacobian is lower triangular by construction

Inverse Autoregressive Flow (IAF)

IAF is the inverse of MAF:

$$x_i = (y_i - \beta_i) \odot \exp(-\alpha_i) \tag{10}$$

Key differences from MAF:

- Fast sampling: Can generate samples in parallel
- Slow density evaluation: Requires sequential computation
- Fast training: When used for variational inference

Training strategy: Use IAF for fast sampling, MAF for fast density evaluation.

TarFlow: Transformer-based Autoregressive Flow

TarFlow is a Transformer-based variant of Masked Autoregressive Flows (MAFs) designed for high-resolution image synthesis.
Key Features:

- Autoregressive Transformer Blocks: Stack of autoregressive Transformer blocks on image patches
- Alternating Autoregression: Alternates autoregression direction between layers
- End-to-End Training: Direct modeling and generation of pixels
- State-of-the-art Performance: First standalone normalizing flow to match diffusion model quality

Applications: High-resolution image synthesis, conditional generation, density estimation.

TarFlow: Architecture and Processing

Overall Architecture:

- Input Processing: Images divided into patches, each patch embedded into D-dimensional vectors
- Transformer Layers: T autoregressive transformer layers with causal attention
- Permutation Strategy: Alternating permutations between layers for bidirectional modeling
- Output: Final latent representation z^T in standard Gaussian space

Layer-wise Processing:

$$x \xrightarrow{f^0} z^1 \xrightarrow{f^1} z^2 \xrightarrow{f^2} \cdots \xrightarrow{f^{T-1}} z^T \tag{11}$$

Each Layer t Consists of:

1. Permutation: $\tilde{z}^t = \pi^t(z^t)$
2. Autoregressive Transformation: $z^{t+1} = f^t(\tilde{z}^t)$

Key Design Principles:

- Causal Attention: Each patch can only attend to previous patches
- Permutation Alternation: Even/odd layers use different permutation orders
- Bidirectional Modeling: Captures dependencies in both directions over multiple layers

TarFlow: Autoregressive Transformation with Permutations

Forward Transformation (Data to Latent):

$$\tilde{z}^t = \pi^t(z^t) \tag{12}$$

$$z_i^{t+1} = \begin{cases} \tilde{z}_i^t & i = 0 \\ (\tilde{z}_i^t - \mu_i^t(\tilde{z}_{<i}^t)) \odot \exp(-\alpha_i^t(\tilde{z}_{<i}^t)) & i > 0 \end{cases} \tag{13}$$

Inverse Transformation (Latent to Data):

$$\tilde{z}_i^t = \begin{cases} z_i^{t+1} & i = 0 \\ z_i^{t+1} \odot \exp(\alpha_i^t(\tilde{z}_{<i}^t)) + \mu_i^t(\tilde{z}_{<i}^t) & i > 0 \end{cases} \tag{14}$$

$$z^t = (\pi^t)^{-1}(\tilde{z}^t) \tag{15}$$

Key Properties:

- Causal Dependencies: μ_i^t and α_i^t depend only on $\tilde{z}_{<i}^t$
- Permutation: π^t alternates between layers for bidirectional modeling
- Autoregressive: Maintains autoregressive structure for exact likelihood

TarFlow: Training Objective and Loss

Complete Training Loss:

$$\min_f 0.5\|z^T\|_2^2 + \sum_{t=0}^{T-1}\sum_{i=1}^{N-1}\sum_{j=0}^{D-1}\alpha_i^t(z_{<i}^t)_j \tag{16}$$

Loss Components:

- Square Term: $0.5\|z^T\|_2^2$ - L2 norm of final latent representation
- Linear Terms: $\sum_{t=0}^{T-1}\sum_{i=1}^{N-1}\sum_{j=0}^{D-1}\alpha_i^t(z_{<i}^t)_j$ - Sum of all scale parameters

Training Process:

- Forward Pass: Transform data x through T layers to get z^T
- Loss Computation: Compute both square term and sum of α parameters
- Backpropagation: Update parameters to minimize the combined loss

Key Insight: The loss simply consists of a square term and a sum of linear terms.

TarFlow: Base Sampling Process

Standard Sampling Procedure:

1. Latent Sampling: Sample $\mathbf{z} \sim p_0$ from prior distribution (e.g., standard Gaussian)
2. Inverse Flow: Apply inverse transformation f^{-1} to get initial sample $\mathbf{y}$

Mathematical Formulation:

$$\mathbf{z} \sim p_0, \quad \mathbf{y} := f^{-1}(\mathbf{z}) \tag{17}$$

Complete Sampling Process:

$$\mathbf{z} \sim \mathcal{N}(0, I) \tag{18}$$

$$\mathbf{y} = f_1^{-1} \circ f_2^{-1} \circ \ldots \circ f_T^{-1}(\mathbf{z}) \tag{19}$$

Key Properties:

- Exact Sampling: No approximation errors in the sampling process
- Deterministic Inverse: Each layer has exact inverse transformation
- Sequential Processing: Must process layers in reverse order

TarFlow: Conditional Generation with Guidance

Guidance Mechanism: Similar to classifier-free guidance in diffusion models.
Guided Reverse Function:

$$\hat{\mathbf{z}}^t = \mathbf{z}^{t+1} \odot \exp(\hat{\boldsymbol{\alpha}}^t(\hat{\mathbf{z}}^t_{<t}; c, w)) + \hat{\boldsymbol{\mu}}^t(\hat{\mathbf{z}}^t_{<t}; c, w) \tag{20}$$

Guided Parameters:

$$\hat{\boldsymbol{\mu}}^t(\hat{\mathbf{z}}^t_{<t}; c, w) = (1 + w)\boldsymbol{\mu}^t(\hat{\mathbf{z}}^t_{<t}; c) - w\boldsymbol{\mu}^t(\hat{\mathbf{z}}^t_{<t}; \emptyset) \tag{21}$$

$$\hat{\boldsymbol{\alpha}}^t(\hat{\mathbf{z}}^t_{<t}; c, w) = (1 + w)\boldsymbol{\alpha}^t(\hat{\mathbf{z}}^t_{<t}; c) - w\boldsymbol{\alpha}^t(\hat{\mathbf{z}}^t_{<t}; \emptyset) \tag{22}$$

Notation:

- c: Class condition (e.g., "cat", "dog")
- $\emptyset$: Unconditional prediction (obtained by dropping class labels)
- w: Guidance weight (controls strength of conditioning)

Remark

The guidance mechanism in TarFlow is directly inspired by classifier-free guidance in diffusion models, providing similar flexibility for conditional generation.

TarFlow: Training for Conditional Generation

Training Strategy:

- Class-conditional Training: Train with class labels c to get $\boldsymbol{\mu}^t(\cdot; c)$ and $\boldsymbol{\alpha}^t(\cdot; c)$
- Unconditional Training: Randomly drop class labels during training to get $\boldsymbol{\mu}^t(\cdot; \emptyset)$ and $\boldsymbol{\alpha}^t(\cdot; \emptyset)$
- Joint Training: Both conditional and unconditional predictions learned simultaneously

Guidance Weight Effects:

- $w = 0$: Standard conditional generation
- $w > 0$: Stronger conditioning, better mode-seeking
- $w < 0$: Weaker conditioning, more diversity

Applications:

- Class-conditional Generation: Generate images of specific classes
- Style Transfer: Control artistic style during generation
- Attribute Control: Control specific attributes (color, shape, etc.)

Advantages over Traditional Methods:

- Flexible Control: Trade-off between diversity and mode-seeking
- No Additional Models: Uses only the trained TarFlow model

Outline

Affine Coupling Layers

Split the input $\mathbf{x}$ into two parts: $\mathbf{x}_A$ and $\mathbf{x}_B$.
Forward transformation:

$$\mathbf{y}_A = \mathbf{x}_A \tag{23}$$

$$\mathbf{y}_B = \mathbf{x}_B \odot \exp(\alpha(\mathbf{x}_A)) + \beta(\mathbf{x}_A) \tag{24}$$

Jacobian:

$$J = \begin{pmatrix} I & 0 \\ \frac{\partial \mathbf{y}_B}{\partial \mathbf{x}_A} & \operatorname{diag}(\exp(\alpha(\mathbf{x}_A))) \end{pmatrix} \tag{25}$$

Determinant: $\det J = \prod_{i \in B} \exp(\alpha_i(\mathbf{x}_A))$

Real NVP: Real-valued Non-volume Preserving

Real NVP uses alternating coupling layers:

1. Checkerboard masking: Alternate which dimensions are transformed
2. Channel-wise masking: For images, alternate RGB channels

Architecture:

- ResNet blocks for α and β networks
- Batch normalization between layers
- Multi-scale architecture for images

Advantages:

- Exact likelihood: Can compute $p(x)$ exactly
- Exact sampling: Can generate samples exactly
- Stable training: No gradient issues

Outline

Neural Ordinary Differential Equations

Instead of discrete transformations, use continuous dynamics:

$$\frac{d\mathbf{x}(t)}{dt} = f_\theta(\mathbf{x}(t), t) \tag{26}$$

Change of variables formula:

$$\log p(\mathbf{x}(t_1)) = \log p(\mathbf{x}(t_0)) + \int_{t_0}^{t_1} \operatorname{tr}\left(\frac{\partial f_\theta}{\partial \mathbf{x}}\right) dt \tag{27}$$

Advantages:

- Flexible architecture: Any neural network can be used
- Adaptive computation: Can adjust number of function evaluations
- Memory efficient: No need to store intermediate states

Proof of Neural ODE Change of Variables Formula

Theorem: For the ODE $\frac{d\mathbf{x}(t)}{dt} = f_\theta(\mathbf{x}(t), t)$, the change of variables formula is:

$$\log p(\mathbf{x}(t_1)) = \log p(\mathbf{x}(t_0)) + \int_{t_0}^{t_1} \text{tr}\left(\frac{\partial f_\theta}{\partial \mathbf{x}}\right) dt \tag{28}$$

Proof

Step 1: Fokker-Planck Equation The probability density $p(\mathbf{x}, t)$ satisfies the continuity equation:

$$\frac{\partial p(\mathbf{x}, t)}{\partial t} + \nabla \cdot (f_\theta(\mathbf{x}, t) p(\mathbf{x}, t)) = 0 \tag{29}$$

Step 2: Expand the Divergence

$$\frac{\partial p(\mathbf{x}, t)}{\partial t} + \nabla \cdot (f_\theta(\mathbf{x}, t)) p(\mathbf{x}, t) + f_\theta(\mathbf{x}, t) \cdot \nabla p(\mathbf{x}, t) = 0 \tag{30}$$

Step 3: Log-Derivative Taking the log-derivative: $\frac{\partial \log p(\mathbf{x},t)}{\partial t} = \frac{1}{p(\mathbf{x},t)} \frac{\partial p(\mathbf{x},t)}{\partial t}$

Proof of Neural ODE Change of Variables Formula (Cont.)

Proof (Cont)

Step 4: Log-Density Evolution

$$\frac{\partial \log p(\mathbf{x}, t)}{\partial t} = -\nabla \cdot f_\theta(\mathbf{x}, t) - f_\theta(\mathbf{x}, t) \cdot \nabla \log p(\mathbf{x}, t) \tag{31}$$

Step 5: Along the Flow Along the flow $\mathbf{x}(t)$, we have:

$$\frac{d \log p(\mathbf{x}(t), t)}{dt} = \frac{\partial \log p(\mathbf{x}, t)}{\partial t} + \frac{\partial \log p(\mathbf{x}, t)}{\partial \mathbf{x}} \cdot \frac{d\mathbf{x}(t)}{dt} \tag{32}$$

Step 6: Substitute the ODE Since $\frac{d\mathbf{x}(t)}{dt} = f_\theta(\mathbf{x}(t), t)$:

$$\frac{d \log p(\mathbf{x}(t), t)}{dt} = -\nabla \cdot f_\theta(\mathbf{x}(t), t) \tag{33}$$

Step 7: Integrate Integrating from t_0 to t_1: $\log p(\mathbf{x}(t_1), t_1) - \log p(\mathbf{x}(t_0), t_0) = -\int_{t_0}^{t_1} \nabla \cdot f_\theta(\mathbf{x}(t), t) dt$.

Proof of Neural ODE Change of Variables Formula (Cont.)

Proof (Cont)

Step 8: Trace Identity For any vector field f, we have:

$$\nabla \cdot f = \text{tr}\left(\frac{\partial f}{\partial \mathbf{x}}\right) \tag{34}$$

Step 9: Final Result Substituting the trace identity:

$$\log p(\mathbf{x}(t_1)) = \log p(\mathbf{x}(t_0)) + \int_{t_0}^{t_1} \text{tr}\left(\frac{\partial f_\theta}{\partial \mathbf{x}}\right) dt \tag{35}$$

Key Insights:

- Continuity Equation: Probability conservation in continuous time
- Trace of Jacobian: Measures volume change in the flow
- Exact Formula: No approximation errors in the continuous case

Computational Challenge: Computing $\text{tr}\left(\frac{\partial f_\theta}{\partial \mathbf{x}}\right)$ is expensive for high dimensions.

FFJORD: Free-form Jacobian of Reversible Dynamics

FFJORD uses the continuous change of variables formula:

$$\log p(\mathbf{x}(t_1)) = \log p(\mathbf{x}(t_0)) + \int_{t_0}^{t_1} \operatorname{tr}\left(\frac{\partial f_\theta}{\partial \mathbf{x}}\right) dt \tag{36}$$

Hutchinson's trace estimator:

$$\operatorname{tr}(A) = \mathbb{E}_{\mathbf{v}}[\mathbf{v}^T A \mathbf{v}] \tag{37}$$

where $\mathbf{v}$ is a random vector with $\mathbb{E}[\mathbf{v}\mathbf{v}^T] = I$.

Remark

We also leverage Hutchinson's trace estimator to compute the trace of the Jacobian matrix in ISM loss in diffusion models.

Benefits:

- Scalable: Works for high-dimensional data
- Flexible: No architectural constraints
- Exact: No approximation errors

Outline

CNFs: Goal & Conditional Likelihood

Problem. Learn a high-dimensional conditional density $p_{\mathbf{Y}|\mathbf{X}}(\mathbf{y} \mid \mathbf{x})$ that is multi-modal and correlated across dimensions.

CNF model. Invertible, $\mathbf{x}$-conditioned map

$$\mathbf{z} = f_\phi(\mathbf{y}; \mathbf{x}), \qquad f_\phi : \mathcal{Y} \times \mathcal{X} \to \mathcal{Z},$$

with a simple conditional base $p_{\mathbf{Z}|\mathbf{X}}(\mathbf{z} \mid \mathbf{x})$ (e.g. diagonal Gaussian).

Change of variables (conditional):

$$p_{\mathbf{Y}|\mathbf{X}}(\mathbf{y} \mid \mathbf{x}) = p_{\mathbf{Z}|\mathbf{X}}\big(f_\phi(\mathbf{y}; \mathbf{x}) \,\big|\, \mathbf{x}\big) \left| \det \frac{\partial f_\phi(\mathbf{y}; \mathbf{x})}{\partial \mathbf{y}} \right|.$$

Training:

$$\max_\phi \; \mathbb{E}_{(\mathbf{x},\mathbf{y})} \left[\log p_{\mathbf{Z}|\mathbf{X}}\big(f_\phi(\mathbf{y}; \mathbf{x}) \mid \mathbf{x}\big) + \log \left| \det \frac{\partial f_\phi(\mathbf{y}; \mathbf{x})}{\partial \mathbf{y}} \right| \right].$$

Sampling: $\mathbf{z} \sim p_{\mathbf{Z}|\mathbf{X}}(\cdot \mid \mathbf{x})$, then $\mathbf{y} = f_\phi^{-1}(\mathbf{z}; \mathbf{x})$.

CNFs: The diagram of CNFs

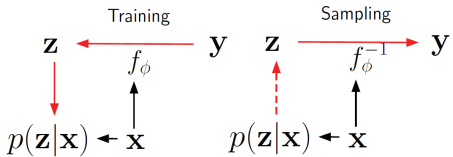

Figure 1: *Diagram of our model in the train and sampling phases. Solid lines represent deterministic mappings and dashed lines represent sampling. The conditioning variable enters the network in base density $p(\mathbf{z}|\mathbf{x})$ and the bijective mappings $f(\mathbf{y}, \mathbf{x})$.*

Conditional Prior	$p(\mathbf{z}\|\mathbf{x}) = \mathcal{N}(\mathbf{z}; \mu(\mathbf{x}), \sigma^2(\mathbf{x}))$
Conditional Split Prior	$p(\mathbf{z}_1\|\mathbf{z}_0, \mathbf{x}) = \mathcal{N}(\mathbf{z}_1; \mu(\mathbf{z}_0, \mathbf{x}), \sigma^2(\mathbf{z}_0, \mathbf{x}))$
Conditional Coupling	$\mathbf{y}_0 = s(\mathbf{z}_1, \mathbf{x}) \cdot \mathbf{z}_0 + t(\mathbf{z}_1, \mathbf{x}); \quad \mathbf{y}_1 = \mathbf{z}_1$

Figure: The diagram of CNFs

CNFs: How to Condition & Build the Flow

Condition injection:

Conditional prior: $p(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}\big(\mu(\mathbf{x}), \operatorname{diag} \sigma^2(\mathbf{x})\big)$.

Coupling (affine): $\begin{cases} \mathbf{y}_a = \mathbf{z}_a, \\ \mathbf{y}_b = \mathbf{z}_b \odot \exp s(\mathbf{z}_a, \mathbf{x}) + t(\mathbf{z}_a, \mathbf{x}), \end{cases} \quad \Rightarrow \quad \log |\det J| = \sum s(\mathbf{z}_a, \mathbf{x}).$

Architecture primitives:

- Coupling layers (additive/affine/rational-quadratic spline) for cheap Jacobians.
- Invertible 1×1 convs or permutations to mix channels.
- Multi-scale (squeeze & factor-out) for efficiency and expressivity.

Implementation pattern. Encode $\mathbf{x}$ once: $\mathbf{h} = g(\mathbf{x})$; pass $\mathbf{h}$ to all conditioner nets (those producing s, t, μ, σ) via concatenation or FiLM/AdaIN-style modulation.

CNFs: Design Choices, Tips, & When to Use CNFs

Design choices.

- Base $p(\mathbf{z} \mid \mathbf{x})$: diagonal Gaussian (fast) vs. low-rank covariance (richer).
- Coupling type: additive (stable), affine (scales variance), spline (captures sharp changes).
- Conditioning nets: small MLP/CNN/Transformer; use weight norm/actnorm for stability.

Training tips.

- Minibatch NLL with EMA of actnorm stats; gradient clipping for deep flows.
- Warm up the scale in affine/spline couplings (e.g., $s \leftarrow \tanh(\cdot)$ or scaled output).
- Multi-scale factor-out improves likelihood and sampling speed.

Why CNFs vs. alternatives?

- Exact conditional likelihood (useful for evaluation & downstream scoring).
- Faster sampling than diffusion; richer joint modeling than per-dimension regressions.
- Good fit for structured prediction where uncertainty matters (SR, optical flow, segmentation logits, etc.).

Outline

Maximum Likelihood Training

Given data $\{\mathbf{x}^{(i)}\}_{i=1}^{N}$, maximize:

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \log p_\theta(\mathbf{x}^{(i)}) \tag{38}$$

Gradient computation:

$$\nabla_\theta \mathcal{L} = \frac{1}{N}\sum_{i=1}^{N} \nabla_\theta \log p_\theta(\mathbf{x}^{(i)}) \tag{39}$$

Challenges:

- Mode collapse: Flow might ignore some data modes
- Training instability: Gradients can be large
- Computational cost: Jacobian computation is expensive

Applications of Normalizing Flows

Generative modeling:

- Image generation (Real NVP, Glow)
- Audio synthesis (WaveGlow)
- Text generation (FlowSeq)

Density estimation:

- Anomaly detection
- Out-of-distribution detection
- Uncertainty quantification

Variational inference:

- Posterior approximation
- Bayesian neural networks
- Variational autoencoders

Outline

Normalizing Flows vs. Other Generative Models

Method	Exact Likelihood	Fast Sampling	Fast Training	Stable
VAE	No	Yes	Yes	Yes
GAN	No	Yes	No	No
Diffusion	No	No	Yes	Yes
Normalizing Flow	Yes	Yes	No	Yes

Unique advantages of Normalizing Flows:

- Exact likelihood: Can compute $p(x)$ exactly
- Exact sampling: Can generate samples exactly
- Latent space: Natural representation learning
- Interpretability: Understandable transformations

Outline

Current Challenges

Computational challenges:

- Jacobian computation: Expensive for high dimensions
- Memory usage: Need to store intermediate states
- Training time: Slower than GANs or VAEs

Modeling challenges:

- Architectural constraints: Must be invertible
- Mode collapse: May ignore some data modes
- Expressiveness: Limited by transformation family

Theoretical challenges:

- Universal approximation: Not guaranteed for all distributions
- Convergence: No global convergence guarantees
- Generalization: Limited theoretical understanding

Future Directions

Architectural improvements:

- More flexible transformations: Beyond coupling layers
- Better conditioning: Improved numerical stability
- Efficient computation: Faster Jacobian computation

Training improvements:

- Better optimization: More stable training
- Regularization: Prevent overfitting
- Multi-scale training: Progressive complexity

Applications:

- High-dimensional data: Images, videos, audio
- Scientific computing: Physics simulations
- Robotics: Motion planning and control

Outline

Summary

Normalizing flows provide:

- Exact likelihood computation
- Exact sampling
- Flexible architectures
- Interpretable representations

Key insights:

- Change of variables: Foundation of all flow-based methods
- Jacobian computation: Central computational challenge
- Architectural design: Balance between expressiveness and efficiency
- Training strategies: Maximum likelihood with proper regularization

Future work:

- Scalability: Handle higher-dimensional data
- Efficiency: Reduce computational costs
- Theoretical understanding: Better convergence guarantees

Welcome inputs

Any comments?

Welcome your inputs!